

Beadandó 2

Gyurcsik Tamás Benjámin

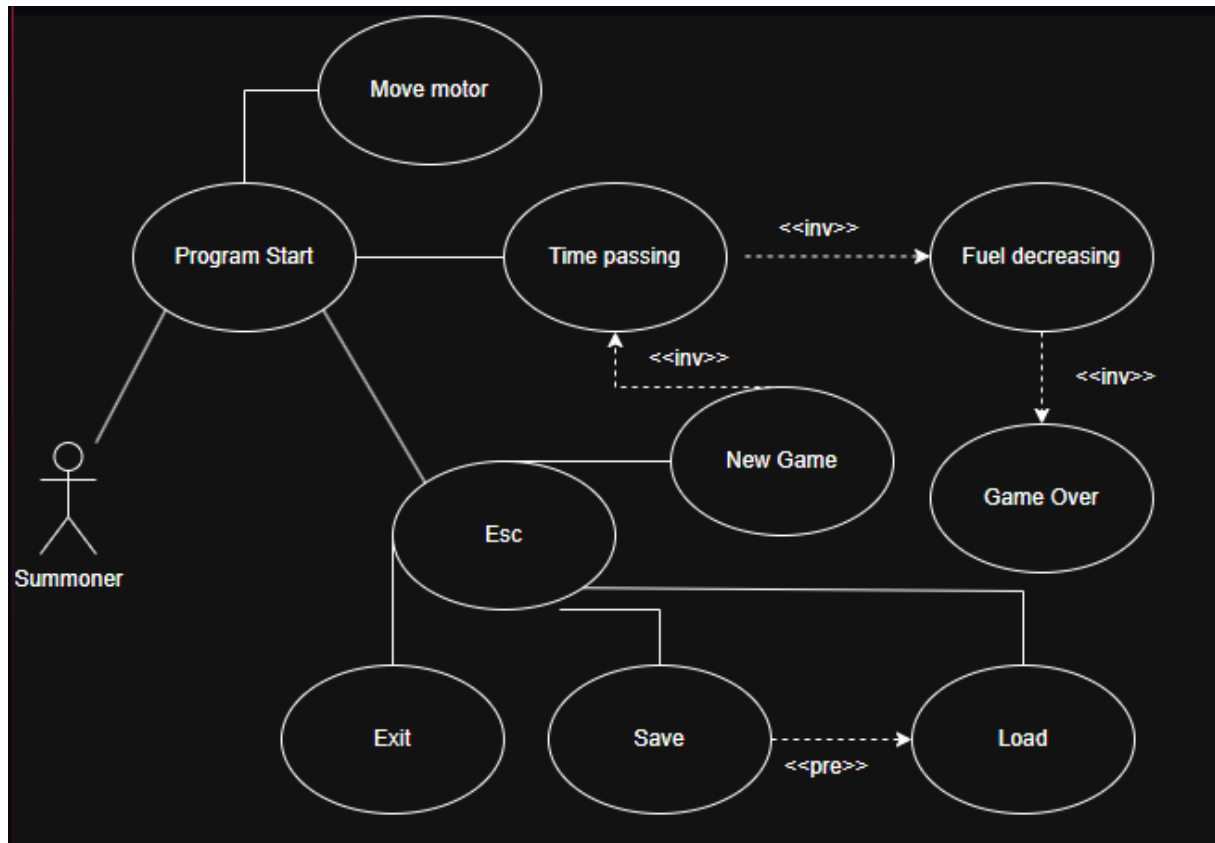
vrn88g@inf.elte.hu

Feladat (9. Gyorsulás):

Készítsünk programot, amellyel az alábbi motoros játékot játszhatjuk. A feladatunk, hogy egy gyorsuló motorral minél tovább tudjunk haladni. A gyorsuláshoz a motor üzemanyagot fogyaszt, egyre többet. Adott egy kezdeti mennyiség, amelyet a játék során üzemanyagcellák felvételével tudunk növelni. A motorral a képernyő alsó sorában tudunk balra, illetve jobbra navigálni. A képernyő felső sorában meghatározott időközönként véletlenszerű pozícióban jelennek meg üzemanyagcellák, amelyek folyamatosan közelednek a képernyő alja felé. Mivel a motor gyorsul, ezért a cellák egyre gyorsabban fognak közeledni, és mivel a motor oldalazó sebessége nem változik, idővel egyre nehezebb lesz felvenni őket, így egyszer biztosan kifogyunk üzemanyagból. A játék célja az, hogy a kifogyás minél később következzen be. A program biztosítson lehetőséget új játék kezdésére, valamint játék szüneteltetésére (ekkor nem telik az idő, és nem mozog semmi a játékban). Ismerje fel, ha vége a játéknak, és jelenítse meg, mennyi volt a játékidő. Ezen felül szüneteltetés alatt legyen lehetőség a játék elmentésére, valamint betöltésére.

Elemzés:

- A feladatot egyablakos asztali alkalmazásként WPF grafikus felülettel valósítjuk meg.
- A program indításakor elindul automatikusan a játék. Kezdetben 100 az üzemanyag, 0 a játékos elért pontja és 0 az eltelt idő.
- Az ablak alsó részén elhelyezünk egy lila téglalapot, ez a motor. Az idő elteltével megjelenik egy fehér csík a képernyő közepén, ez a "pálya". Illetve a véletlenszerűen generált rózsaszín négyzetek az üzemanyagok.
- Amikor a motor és az üzemanyag összeütközik, akkor az üzemanyag begyűjtésre kerül. Ez azt jelenti hogy a motor 'Fuel' értékéhez adunk 10-et és a négyzet eltűnik.
- Az idő elteltével egyre gyorsabban közelednek a képernyő teteje felől lefelé az üzemanyagok, emiatt elég eltelt idő után már lehetetlen lesz felszedni őket. Ez biztosítja hogy a játék véget fog érni.
- Az 'Esc' gomb lenyomásával előjön egy menu. A 'New Game' gomb egy teljesen új játékot indít, lenullázza az eddigi adatainkat. Az 'Exit' gomb bezárja a játékot. A 'Save' készít egy mentést, szöveges dokumentum formájában. A 'Load' lehetővé teszi a mentés betöltését.
- A játék végekor megáll minden mozgás, ekkor az eltelt idő továbbra is leolvasható. Innen az 'Esc' billentyű megnyomásával navigálhatunk tovább.



1. ábra: Felhasználói eset diagram

Tervezés:

Szerkezet:

- A programot MVVM architektúrában valósítjuk meg, ennek megfelelően View, Model, ViewModel és Persistence névtereket valósítunk meg az alkalmazáson belül. A program környezetét az alkalmazás osztály (App) végzi, amely példányosítja a modellt, a nézetmodellt és a nézetet, biztosítja a kommunikációt, valamint felügyeli az adatkezelést.
- A játék logikája és a mentés az első beadandónál átemelt, érintetlen Model és Persistence mappában helyezkedik el
- A megjelenítést a View és a ViewModel végzi

Perzisztencia:

- A betöltés és a mentés feladatokat látja el
- A *FileManager.cs* valósítja meg a projektben. Ennek az osztálynak két metódusa van: *Load*, *Save*. Az *IFileManager*-ből származtatott. Ennek célja az hogy mockolhassuk tesztelés során.

Model:

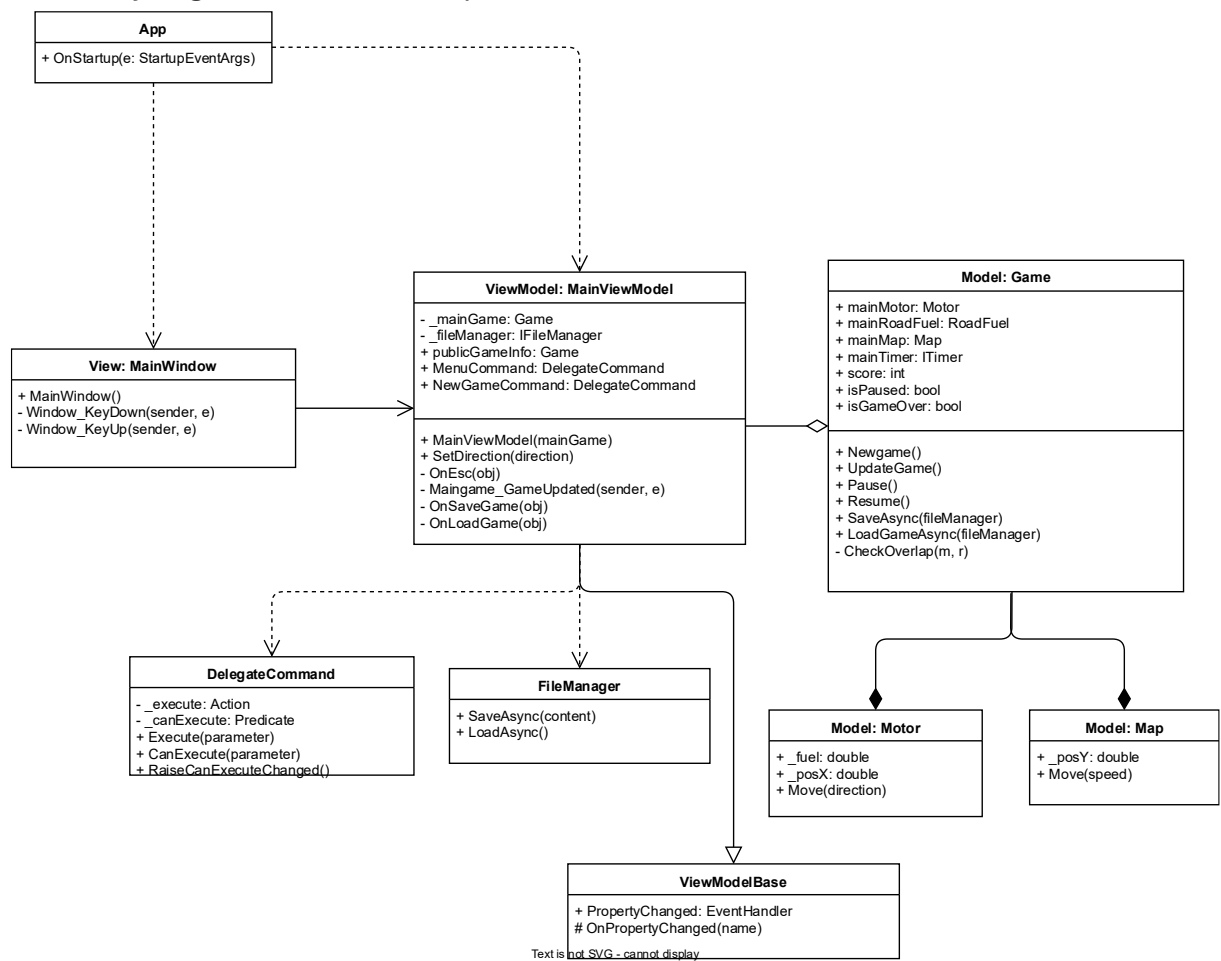
- A teljes játék logikája, működése ebben a mappában helyezkedik el.
- A játék maga, meglepő módon, a *Game.cs* osztályban folyik. Ebben valósul meg a játék adatainak kiszámítása amely három fő osztályt használ fel ehhez:
 - *Motor.cs*: A motor felelős a saját mozgásáért és az üzemanyag kiszámításáért.
 - *RoadFuel.cs*: Felel a saját mozgásáért és a begyűjtésének kezeléséért.
 - *Map*: Felel a saját mozgásáért.
- Az idő kezelést a *View*-tól való teljes függetlenség miatt egy saját időzítővel kezeljük. Ez a *FinalTimer.cs* amely az *ITimer.cs*-ből származtatott, szintén a mock-olás miatt.
- Az *Enums.cs*-ben található meg a 3 irány amelyet a motor felhasznál a mozgás során.
- A *CollectEventArgs.cs* és a *LoadEventArgs.cs* két saját EventArgs, amelyek a saját eseményeink kezelésére kelljenek.
- A *Game.cs*-ben minden egyéb logika megtalálható:
 - A mentés és betöltés itt kezelendő egy *FileManager* segítségével
 - Az új játék indítása is itt valósul meg. A betöltés egyszerűbb kezelése miatt a *NewGame* metódus "overloadol"-ásra került
 - A *Resume* és *Pause* a játék helyes megállítását és folytatását teszi lehetővé. Főként az óidőkezelés miatt lett bevezetve.
 - Az *UpdateGame* a legfontosabb metódus. Az adatok frissítését csinálja.

Nézet:

- A nézetet a *MainWindow.xaml* és a *MainViewModel.cs* biztosítja. Az előbbi az utóbbitól kapott *publicGameInfo* nevű objektumon keresztül kapja meg a megjelenítéshez szükséges adatokat.
- A játéktáblán a következő található (*MainWindow.xaml*):
 - Grid: minden ezen belül helyezkedik el
 - Viewbox és Canvas: a számolás relatív koordinátákkal történik. Ahhoz hogy ezt ne kelljen megváltoztatni egy 1x1-es canvast nyújtottam ki a viewbox segítségével.
 - Rectangle: az adott játékelemet szimulálja, motor/fuel/map
 - StackPanel és TextBlockok: ez a UI, innen olvasható le a score, fuel és az idő.
 - Border: tartalmazza a menüt, felel a láthatóságért is. A játék szüneteltetésekor elszűrkíti aképernyőt.
 - StackPanel: tartalmazza a menüt
 - Button: a különböző gombokat jelentik, rájuk kattintva lefut a megfelelő funkció

- A MainViewModel működése:
 - Ezen az osztályon belül deklarálásra került egy privát Game és egy publikus publicGameInfo. Az utóbbi az ami szállítja az adatot a MainWindownak. Továbbá különböző DelegateCommandok is.
 - A MainViewModel konstruktorában inicializáljuk a _mainGame-t, ez a játék logikája, nem érhető el a View-ből közvetlen. Továbbá ezt a commandokkal is megtesszük és feliratkozunk a játékfrissítő eseményre. Ezután elindul a játék automatikusan.
 - Az osztályban egy "command check" található. Ez mondja meg, hogy az adott command futtatható-e. Továbbá, a commandok tényleges funkciói is itt kerültek megírásra.

Az osztálydiagrammot az alábbi kép tartalmazza:



Tesztelés:

- A modell funkcionalitása egységtesztek segítségével lett ellenőrizve a **Gyorsulás_Test** osztályban.
- A fájlok kezelését *IFileManager* segítségével tesszük meg. Ez lehetővé teszi a mock-olást a tesztek során.
- Hasonló elképzelés miatt lett bevezetve az *ITimer* is, azonban az *UpdateGame* metódus meghívásával szimulálhatjuk az idő múlását, így nem szükséges ténylegesen időzítőt használni.
- Tesztesetek:
 - *NewGame_ShouldResetValues*: Vizsgálja, hogy új játék során a játék a kezdeti értékekkel indul-e.
 - *UpdateGame_ShouldIncreaseElapsedTime*: Vizsgálja, hogy mozognak-e a megfelelő objektumok.
 - *UpdateGame_ShouldConsumeFuel*: Vizsgálja, hogy az üzemanyag ténylegesen fogy-e.
 - *Collision_ShouldIncreaseScore_AndFuel*: vizsgálja, hogy ütközés során kapunk-e pontot és üzemanyagot.
 - *Save_ShouldWriteAllData*: Vizsgálja, hogy a mentés kiír-e adatokat a txt-be
 - *LoadGame_ShouldRestoreGameState*: Vizsgálja, hogy a betöltés után a betöltendő adatok szerepelnek-e.